

Premium Tees

Complete project guidelines: architecture, features, roles, payments, CMS, homepage templates, components, environment setup, and future maintenance tasks.

Stack: Next.js 15 · React 19 · Prisma · Postgres · Auth.js · Razorpay (INR) · Resend · Vercel

Live: <https://premium-tees.vercel.app>

Repo: <https://github.com/AnishWebDev/premium-tees>

Document date: 31 July 2026 · Version aligned with commit series including homepage template redesign

Contents

1. Project overview
2. Tech stack & scripts
3. Local & production setup
4. Environment variables
5. Demo accounts & roles
6. Feature inventory (storefront)
7. Feature inventory (admin)
8. Checkout, payments & fulfillment
9. Email & notifications
10. CMS & site content
11. Theme system
12. Homepage templates
13. Home building blocks
14. Effects library
15. Component map (where to change what)
16. App routes & API map
17. Database models
18. Accessibility & Lighthouse guidelines
19. Pending / recommended tasks
20. How-to recipes (common edits)
21. Git & deployment notes

1. Project overview

Premium Tees is a production-oriented apparel e-commerce app focused on the Indian market (INR, Razorpay). It includes a public storefront, customer accounts, and a role-gated admin CMS for catalog, orders, returns, content, and visual style.

What it sells

Premium t-shirts / apparel with variants (size, color), inventory, reviews, wishlist, and coupons.

What makes it configurable

CMS JSON content, theme presets, and SuperAdmin-selectable homepage templates that reuse shared blocks.

2. Tech stack & scripts

Layer	Technology
Framework	Next.js 15.5 (App Router, Turbopack in dev)
UI	React 19, Tailwind CSS 4, Radix UI, Lucide icons
Language	TypeScript 5
Database	PostgreSQL (Neon in production) + Prisma 6
Auth	Auth.js / NextAuth v5 (JWT), credentials + optional Google
Payments	Razorpay (UPI / cards), demo checkout fallback
Email	Resend (invoices, shipped, returns)
Media	Cloudinary (optional admin uploads) or external image URLs
State	Zustand (cart), TanStack Query
Forms	react-hook-form + Zod
Motion	framer-motion (effects / reveals)
Hosting	Vercel

npm scripts

Script	Purpose
<code>npm run dev</code>	Local development server
<code>npm run build</code>	prisma generate + next build
<code>npm run db:generate</code>	Generate Prisma client
<code>npm run db:push</code>	Push schema to database
<code>npm run db:seed</code>	Seed products, users, content
<code>npm run db:studio</code>	Prisma Studio

3. Local & production setup

Local

```
cp .env.example .env
# fill DATABASE_URL, AUTH_SECRET, and either Razorpay test keys or DEMO_CHECKOUT=true
npm install
npm run db:push
npm run db:seed
npm run dev
```

Production (Vercel)

1. Connect the GitHub repo to Vercel.
2. Set all required env vars (see §4). Use live Razorpay keys for real customers.
3. Set `DEMO_CHECKOUT=false` and `NEXT_PUBLIC_DEMO_CHECKOUT=false`.
4. Run schema push + seed once against production Postgres if empty.
5. Configure Razorpay webhook → `https://your-domain.com/api/webhooks/razorpay` (event: `payment.captured`).
6. Verify Resend sender domain for invoices.

One-time price conversion

If seeded prices still look USD-like (~38–85), run:

```
DATABASE_URL="..." npx tsx scripts/convert-prices-to-inr.ts
```

4. Environment variables

Canonical template: `.env.example`

Variable	Required	Notes
<code>DATABASE_URL</code>	Yes	Postgres connection string
<code>AUTH_SECRET</code>	Yes	<code>openssl rand -base64 32</code>
<code>AUTH_URL</code>	Recommended	App origin, no trailing slash
<code>NEXT_PUBLIC_APP_URL</code>	Recommended	Public site URL
<code>RAZORPAY_KEY_ID</code>	Live pay	Must start with <code>rzp_</code>
<code>RAZORPAY_KEY_SECRET</code>	Live pay	Server secret
<code>NEXT_PUBLIC_RAZORPAY_KEY_ID</code>	Live pay	Same as key id (client)
<code>RAZORPAY_WEBHOOK_SECRET</code>	Live pay	Webhook HMAC
<code>RESEND_API_KEY</code>	Emails	Invoices / shipped / returns
<code>EMAIL_FROM</code>	Emails	Verified sender
<code>ORDER_NOTIFY_EMAIL</code>	Recommended	BCC of every invoice
<code>DEMO_CHECKOUT</code>	Dev	<code>true</code> allows paid demo without Razorpay
<code>NEXT_PUBLIC_DEMO_CHECKOUT</code>	Dev	Shows demo UI banner
<code>CLOUDINARY_*</code>	Optional	Admin image uploads
<code>NEXT_PUBLIC_APP_NAME</code>	Optional	Defaults to “Premium Tees”
<code>AUTH_GOOGLE_ID/SECRET</code>	Optional	Supported in code; not listed in <code>.env.example</code>

Production warning: Never leave `DEMO_CHECKOUT=true` on a live store. Without Razorpay and with demo off, checkout returns 503.

5. Demo accounts & roles

Created by `npm run db:seed`. Role enum: `USER` | `ADMIN` | `SUPERADMIN`.

Role	Email	Password	Access
SuperAdmin	<code>superadmin@premiumtees.com</code>	<code>SuperAdmin123!</code>	All admin + staff roles, hard-delete products, homepage template picker, system status
Admin	<code>admin@premiumtees.com</code>	<code>Admin123!</code>	Catalog, orders, returns, content, theme/style (no staff / hard delete / template switch)
Guest (USER)	<code>guest@premiumtees.com</code>	<code>Guest1234!</code>	Shop, cart, checkout, wishlist, orders, return requests

Where roles are enforced

- `src/middleware.ts` — gates `/admin/*`, `/profile/*`, `/orders/*`
- `src/lib/roles.ts` + `src/lib/auth.ts` — `requireAdmin()`, `requireSuperAdmin()`
- Homepage template mutation — SuperAdmin only (API + content editor)

6. Feature inventory (storefront)

Feature	Description	Primary paths
Homepage templates	5 distinct layouts selectable in CMS	<code>/</code> , <code>src/components/home/templates/</code>
Shop / catalog	Filter, sort, category browse	<code>/shop</code> , <code>/collections</code>
Product detail	Gallery, variants, reviews, related, recently viewed	<code>/product/[slug]</code>
Cart	Zustand cart with inventory-aware lines	<code>/cart</code> , <code>src/lib/stores/cart-store.ts</code>
Checkout	Guest or signed-in; coupons; India shipping; Razorpay / demo	<code>/checkout</code>
Wishlist	Auth-required save for later	<code>/wishlist</code>
Account orders	History, detail, return request	<code>/orders</code>
Content pages	About, FAQ, contact, shipping, privacy, terms	<code>/about</code> , <code>/faq</code> , ...
Newsletter	Email capture → Newsletter model	Home blocks / footer
Theme application	CSS variables from CMS theme	<code>ThemeStyle</code>

Shipping (INR)

Configured in `src/lib/constants.ts` — example tiers: ₹79 / ₹149 / ₹249, free shipping threshold typically ₹1999.

7. Feature inventory (admin)

Area	URL	Capabilities
Dashboard	/admin	Overview stats
Products	/admin/products	CRUD, featured/bestseller/new flags, images, variants
Categories	/admin/categories	Manage collections
Inventory	/admin/inventory	Stock levels / low stock
Orders	/admin/orders	Status, tracking, packing slip print
Returns	/admin/returns	Approve / reject / refund workflow
Customers	/admin/customers	Customer list
Coupons	/admin/coupons	Percent / fixed discounts
Reports	/admin/reports	Analytics
Site content	/admin/content	Hero, home, about, FAQ, IG, newsletter, template
Settings / style	/admin/settings	Theme editor, password, system status
Staff	/admin/staff	SuperAdmin role management

8. Checkout, payments & fulfillment

Flow

1. Client posts cart to `POST /api/checkout`.
2. Server creates `Order` with status `PENDING`, currency `inr`.
3. **Demo path:** if `DEMO_CHECKOUT=true` and Razorpay keys missing → invent demo payment IDs, call `fulfillOrder`, redirect to success.
4. **Live path:** create Razorpay order (paise), return key + order id to `Checkout.js`.
5. Client verifies signature via `/api/checkout/verify` → `fulfillOrder`.
6. Webhook `payment.captured` is a backup fulfillment path.

Key files

- `src/lib/razorpay.ts`
- `src/lib/fulfill-order.ts` — mark PAID, decrement inventory, coupon use, invoice email
- `src/components/checkout/checkout-form.tsx`
- `src/app/api/checkout/`, `src/app/api/webhooks/razorpay/`

Ops after payment

- **Packing slip:** Admin → Orders → Pack / print page
- **Shipped email:** set status Shipped + carrier/tracking
- **Returns:** customer requests from order detail; admin reviews at `/admin/returns`

9. Email & notifications

Email	Trigger	File
Order invoice / confirmation	Payment fulfilled	src/lib/email/order-confirmation.ts
Shipped notification	Admin marks shipped	src/lib/email/shipped-notification.ts
Return updates	Return status changes	src/lib/email/return-notification.ts

Without Resend keys, fulfillment still succeeds; emails are skipped with a console warning.

10. CMS & site content

Content is stored in Prisma model `SiteContent` as JSON keyed blobs. Defaults live in `src/lib/site-content.ts` (`DEFAULT_SITE_CONTENT`). Admin UI: `src/components/admin/site-content-editor.tsx` at `/admin/content`.

Key	Fields / purpose
<code>site</code>	name, description
<code>hero</code>	brand, headline, subheadline, imageUrl, videoUrl, CTAs
<code>home</code>	template, marqueeItems, essentials, story, bestSellers, newArrivals, categories
<code>about</code>	story paragraphs, values, CTA
<code>testimonials</code>	title, subtitle, items
<code>faq</code>	title, Q&A items
<code>instagram</code>	title, profileUrl, images[]
<code>newsletter</code>	title, subtitle
<code>theme</code>	full ThemeData object

Mid-page story copy is always CMS-driven via `home.story`. Templates choose how to present it (parallax band, sticky split, mission statement, etc.), not different copy sources.

11. Theme system

Files: `src/lib/theme.ts` , `src/components/theme/theme-style.tsx` , `src/components/admin/theme-editor.tsx` , `src/lib/fonts.ts` .

Theme writes CSS variables (`--background` , `--foreground` , button/link vars, font families).

Presets

`studio` , `editorial` , `slate` , `olive` , `noir` , `ink-gold`

Editable fields

- Colors: background, foreground, muted, border, accent, ring
- Fonts: sans + display (from FONT_CATALOG)
- Buttons: radius (pill/rounded/square), style (solid/outline/soft), weight
- Links: underline / accent / subtle / bold

Theme affects the whole storefront chrome. Homepage templates change layout/density, not necessarily the color preset (unless you also change Theme).

12. Homepage templates

Registry: `src/lib/home-templates.ts`

Router: `src/components/home/templates/index.tsx`

Page data loader: `src/app/(store)/page.tsx`

ID	Name	Density	Inspired by	Structure summary
<code>parallax</code>	Parallax	sparse	Apple / campaigns	Cinematic hero → marquee → chapter bands → look tiles. No newsletter.
<code>simple</code>	Simple	balanced	Everlane	Static split hero → value pillars → 4-up grid → text categories → mission → testimonials → light signup.
<code>editorial</code>	Editorial	sparse	SSENSE	Type masthead → feature drop → quote → product rows → sticky story → FAQ → band newsletter.
<code>lookbook</code>	Lookbook	balanced	Aritzia	Carousel → mosaic → stacked panels → oversized looks → Instagram. Image-first, no newsletter.
<code>commerce</code>	Commerce	dense	Nike	Media hero → promo → pills → bestsellers shelf → featured grid → new arrivals → categories → IG → signup.

How to switch template

1. Sign in as SuperAdmin.
2. Go to **Admin** → **Site content** → **Home**.
3. Pick a template card (shows density + inspiration).
4. Save content.

How to add a new template

1. Add id to `HOME_TEMPLATE_IDS` and metadata in `src/lib/home-templates.ts`.
2. Create `src/components/home/templates/your-home.tsx` accepting `HomeTemplateProps`.
3. Register it in `src/components/home/templates/index.tsx`.
4. Reuse blocks from `src/components/home/blocks/` and effects from `src/components/effects/`.
5. Keep a11y: headings, focus rings, contrast overlays, reduced motion.

13. Home building blocks

Directory: `src/components/home/blocks/`

Component	File	Use when
StaticHero	static-hero.tsx	Calm split hero, no parallax/video
MediaHero	media-hero.tsx	Shorter commerce media hero ± video
EditorialMasthead	editorial-masthead.tsx	Type-only hero (no image)
ChapterBand	chapter-band.tsx	Full-viewport cinematic chapter
ValuePillars	value-pillars.tsx	3 text value columns
MissionStatement	mission-statement.tsx	Plain-text mission (no image)
FeatureDrop	feature-drop.tsx	One oversized product feature
ProductShelf	product-shelf.tsx	Horizontal product shelf
ProductRowList	product-row-list.tsx	Magazine product index rows
CategoryPills	category-pills.tsx	Quick category nav
PromoBanner	promo-banner.tsx	High-contrast promo strip
LookTile	look-tile.tsx	Image-first look (minimal price chrome)
ImageMosaic	image-mosaic.tsx	Asymmetric collage
StackedPanels	stacked-panels.tsx	Full-bleed stacked collection panels
InlineNewsletter	inline-newsletter.tsx	Compact light/band signup

Shared home components (still used)

- `hero.tsx` — cinematic parallax/video hero
- `home-story.tsx` — parallax or sticky-split story
- `featured-products.tsx` , `categories-section.tsx`
- `testimonials.tsx` , `instagram-gallery.tsx` , `newsletter-section.tsx`

14. Effects library

Barrel: `src/components/effects/index.ts`

Effect	File	Notes
Carousel	carousel.tsx	ARIA carousel; pauses on hover/focus; respects reduced motion
ParallaxImage / ParallaxSection	parallax.tsx	Scroll-linked background motion
VideoBackground	video- background.tsx	Muted loop; poster fallback when reduced motion
Reveal	reveal.tsx	In-view fade/slide
Marquee	marquee.tsx	Ticker; SR-friendly; motion-safe animation
StickySplit	sticky-split.tsx	Sticky image + scrolling copy
HorizontalScroll	horizontal-scroll.tsx	Snap scroller with keyboard focus region

15. Component map — where to change what

If you want to...	Change / add here
Edit homepage layout structure	<code>src/components/home/templates/*-home.tsx</code>
Add a reusable home section	<code>src/components/home/blocks/</code>
Change hero copy / image / video	Admin → Content → Hero (or <code>site-content.ts</code> defaults)
Change site colors / fonts / buttons	Admin → Settings theme editor (<code>theme.ts</code>)
Change product card look	<code>src/components/product/product-card.tsx</code>
Change header / nav / footer	<code>src/components/layout/header.tsx</code> , <code>footer.tsx</code>
Change checkout UX	<code>src/components/checkout/checkout-form.tsx</code>
Change shipping rates	<code>src/lib/constants.ts</code>
Change roles / access	<code>src/lib/roles.ts</code> , middleware, admin staff UI
Change fulfillment / inventory side-effects	<code>src/lib/fulfill-order.ts</code>
Change email HTML	<code>src/lib/email/*</code>
Add admin page	<code>src/app/(admin)/admin/...</code> + sidebar in <code>admin-sidebar.tsx</code>
Add API endpoint	<code>src/app/api/...</code>
Change DB schema	<code>prisma/schema.prisma</code> then <code>db:push</code>
Seed demo data	<code>prisma/seed.ts</code> , <code>scripts/*</code>

Major folders

<code>src/app/(store)/</code>	Public storefront pages
<code>src/app/(account)/</code>	Profile & orders
<code>src/app/(auth)/</code>	Login / register
<code>src/app/(admin)/admin/</code>	Admin UI
<code>src/app/api/</code>	Route handlers
<code>src/components/</code>	UI by domain (home, product, admin, effects...)
<code>src/lib/</code>	Business logic, CMS, theme, auth, email
<code>prisma/</code>	Schema + seed
<code>scripts/</code>	One-off maintenance scripts
<code>docs/</code>	This handbook

16. App routes & API map

Storefront routes

`/`, `/shop`, `/product/[slug]`, `/collections`, `/cart`, `/checkout`, `/checkout/success`, `/wishlist`,
`/about`, `/contact`, `/faq`, `/shipping`, `/privacy`, `/terms`

Account

`/profile`, `/orders`, `/orders/[id]`

Auth

`/login`, `/register`

Admin

`/admin`, products, categories, inventory, orders (+ print), returns, customers, staff, coupons, reports, content, settings

Key APIs

- Auth: `/api/auth/[...nextauth]`, `/api/auth/register`
- Catalog: products, categories, reviews, coupons, wishlist, upload, newsletter
- Checkout: `/api/checkout`, `/api/checkout/verify`, `/api/webhooks/razorpay`
- Orders / returns: `/api/orders`, `/api/orders/[id]/return`
- Admin: products, categories, orders, returns, customers, staff, coupons, content, analytics, password

17. Database models

Schema: `prisma/schema.prisma`

Model	Purpose
User / Account / Session / VerificationToken	Auth.js + roles
Category, Product, ProductImage, Variant, Inventory	Catalog
CartItem	Persisted / session cart lines
Order, OrderItem	Checkout + fulfillment (Razorpay ids; legacy Stripe fields unused)
ReturnRequest	Customer returns workflow
Review, WishlistItem, Address	Engagement / account
Coupon, Newsletter	Growth / merchandising
SiteContent	CMS JSON by key

Order statuses include PENDING → PAID → ... → SHIPPED / REFUNDED (see enum in schema).

18. Accessibility & Lighthouse guidelines

Applied especially to homepage templates / effects. Follow these when adding UI:

- One clear `h1` per page; nest `h2` / `h3` logically.
- Interactive elements need visible focus rings (`focus-visible:ring-...`).
- Text on imagery needs dark overlays; prefer ~white/90 on dark scrims.
- Decorative images: `alt=""` when adjacent text already names the content.
- Respect `prefers-reduced-motion` (carousel autoplay off, video → poster, marquee animation off).
- Limit `priority` images (usually hero + 1 product) to protect LCP.
- Regions: use `aria-label` / `aria-labelledby` on carousels, shelves, nav groups.
- Don't hide essential CTAs behind hover-only styles (touch + SR users).

19. Pending / recommended tasks

Operational and polish items that are not fully closed:

Priority	Task	Notes
P0	Configure live Razorpay keys + webhook	Disable demo checkout before real traffic
P0	Configure Resend + verified EMAIL_FROM	Invoices + BCC to ops inbox
P1	Confirm production DATABASE_URL backups	Neon / provider snapshots
P1	Cloudinary (optional) for admin uploads	Otherwise use external image URLs
P2	Document / wire Google OAuth env vars	Supported in auth code
P2	Remove unused Stripe fields on Order	Schema cleanup
P2	Align Address default country to IN	Schema still defaults US
P2	Content polish (INR language in testimonials)	Seed still has “dollar” phrasing
P3	Add automated tests for checkout / roles	Currently ops-validated
P3	Per-template CMS fields (optional)	Today templates share home.* copy
P3	Performance pass on commerce template	Most product-dense; watch image weight
P3	Custom domain + AUTH_URL alignment	Must match production URL

20. How-to recipes (common edits)

A. Change homepage hero image / headline

1. Admin → Site content → Hero
2. Update image URL (and optional video URL)
3. Edit headline / CTAs → Save

B. Change which products appear on home

1. Admin → Products → edit flags: Featured / Best seller / New arrival
2. Homepage queries these flags (`getFeaturedProducts` , `getBestSellers` , `getNewArrivals`)

C. Add a new home block and use it in Commerce

1. Create `src/components/home/blocks/my-block.tsx`
2. Import into `commerce-home.tsx` and place in JSX
3. If copy is needed, extend `HomeData` in `site-content.ts` + admin editor fields

D. Add a theme preset

1. Append to `THEME_PRESETS` in `src/lib/theme.ts`
2. It appears automatically in the theme editor

E. Add an admin-only page

1. Add route under `src/app/(admin)/admin/...`
2. Link it in `src/components/admin/admin-sidebar.tsx`
3. Protect mutations with `requireAdmin()` / `requireSuperAdmin()`

F. Go live checklist

1. `DEMO_CHECKOUT=false` , public demo flag false
2. Live Razorpay keys + webhook secret
3. Resend configured; send a test order
4. `AUTH_URL` / `NEXT_PUBLIC_APP_URL` match domain
5. Change seeded admin passwords
6. Smoke-test: shop → checkout → verify → packing slip → ship → return

21. Git & deployment notes

- Primary branch: `main` (auto-deploy to Vercel).
- Prefer few, purposeful commits for Hobby / cleaner history.
- Git author for Vercel Hobby deployments has used: `Anish <r.s.anish88@gmail.com>`.
- Never commit secrets (`.env` , `.env.vercel.local`).
- After schema changes, run `db:push` (or migrations) against the same DB Vercel uses.

Useful file index

```
README.md
.env.example
prisma/schema.prisma
src/middleware.ts
src/lib/auth.ts
src/lib/roles.ts
src/lib/razorpay.ts
src/lib/fulfill-order.ts
src/lib/site-content.ts
src/lib/theme.ts
src/lib/home-templates.ts
src/components/effects/*
src/components/home/blocks/*
src/components/home/templates/*
src/app/(store|admin|account|auth)/**
src/app/api/**
docs/premium-tees-project-handbook.html
docs/Premium-Tees-Project-Handbook.pdf
```

End of handbook — regenerate the PDF from the HTML source if this document is updated: open `docs/premium-tees-project-handbook.html` and print to PDF, or re-run the Chrome headless print command in the repo docs folder.